ASPHALTPCI PRO · ESPECIFICACIÓN DE DISEÑO

# Sección «Recomendaciones» de la app

Panel del agente: calibrar, completar datos, generar y ajustar. Diseño para implementar en la app web (React 19 + TypeScript)

| | |
|---|---|
| Versión | 3.0 · 2026-11-04. Reemplaza a la 2.0: el panel deja de ser un asistente aparte de 7 pasos y pasa a ser una sección de la app, a continuación de Resultados, con una sola pantalla de configuración y una pantalla de resultado con «Editar parámetros» |
| Destinatario | Codex (sesión de implementación); revisión por Claude |
| Interfaz de referencia | La app que corre en `localhost:5000`: la copia local `repositorios/Asphalt-pci_mapa` (menú lateral con los pasos del proyecto, pantalla Resultados, componentes `components/ui`, tokens de `index.html`, almacenamiento en `services/localApi.ts`). Los números de línea citados corresponden a esa copia |
| Repositorio en GitHub | `PCI-Project/PCI-Proyecto-Tesis`: mismos componentes, pero guarda por `axios` en un backend Express + MongoDB. El diseño funciona con los dos almacenamientos (sección 6) |
| Agente (ya hecho) | Paquete `agente-recomendaciones`: motor, redactor con Ollama y servicio FastAPI que devuelve solo datos (`INTEGRACION.md`) |
| Documentos de apoyo | `docs/01-contrato-entrada-salida.md`, `schemas/*.json`, `catalogo/*.json`, `ejemplos/entrada_ejemplo.json` y `docs/panel/informe-referencia.pdf` (cómo debe verse el informe) en el paquete del agente |

**ALCANCE DE LA TAREA**

Una sección nueva de la app, **Recomendaciones** (paso 5, después de Resultados), con: configuración de lo que el agente necesita (datos que faltan, equipo disponible, costos, presupuesto y forma de priorizar), el botón que pone a trabajar al agente, y la pantalla de **resumen e informe** dibujados en la web con exportación a PDF y la opción **Editar parámetros** para ajustar la consulta y volver a generarla. Incluye los cambios mínimos de guardado (sección 6). El agente solo devuelve datos; no se modifica su lógica.

# 0 Instrucciones para la implementación

## Qué construir

Una sección nueva de la app, **Recomendaciones**, que aparece como **paso 5** del proyecto, justo después de Resultados (vista `AGENT`). Es **una sola pantalla con dos modos**:

- **Configurar:** el inspector calibra las opciones (equipo disponible, costos, presupuesto, forma de priorizar), completa los datos que la evaluación PCI no tiene (tipo de vía, bordillo, tránsito, edad del pavimento, drenaje…) y pone a trabajar al agente con un botón.
- **Resultado:** con la respuesta `{analisis, narrativa}` del agente, la web dibuja **un resumen en pantalla y el informe completo** (exportable a PDF). Un botón **Editar parámetros** devuelve al modo Configurar con todo lo ingresado, para ajustar la consulta y volver a generarla; el resultado anterior se conserva y se puede comparar.

El agente no conversa: recibe `entrada.json` (según `schemas/entrada.schema.json`) y devuelve datos. El informe y el resumen los arma la web.

## Reglas de trabajo

1. **Dónde:** en la copia local `repositorios/Asphalt-pci_mapa` (la que se ve en `localhost:5000`), en una rama `feature/recomendaciones`. Esa copia tiene cambios de Dk sin guardar (`public/manifest.json` y las traducciones `de, en, es, fr`): **no los incluyas** en tus commits ni los reviertas. **No hacer commit, push, PR ni merge** sin confirmación de Dk: Claude revisa el diff y el navegador primero. Dk la arranca con `iniciar-aplicacion.cmd` (`vite --port 5000`); no detengas ese proceso.
2. No modificar la lógica del agente. Reemplazar `agent/` del repositorio de GitHub por el paquete del agente es una tarea aparte (6.5); la sección solo consume su API.
3. **Sin dependencias nuevas.** Usar React, Tailwind (CDN con los tokens de `index.html`), jsPDF + autoTable (ya cargados por CDN en `index.html`, como en `services/exportService.ts`) y los componentes de `components/ui` (`Card`, `Button`, `Input`, `InfoTip`, `Modal`, `Tooltip`). Las llamadas al servicio del agente usan `fetch` (la copia local no tiene `axios` en `package.json`; el proyecto se guarda con `services/localApi.ts`).
4. Todos los textos de la interfaz por `t()`, con claves nuevas en `translations/es.ts` y `translations/en.ts`: `stepper.step5` y el bloque `agent.*`. Los demás idiomas caen al inglés (`hooks/useTranslations.ts`). Los textos del informe que vienen en `narrativa` se muestran tal cual, en español.
5. Tema claro y oscuro: solo clases de tokens (`bg-bg-surface`, `text-text-base`, `border-border-base`, `bg-primary`, `text-danger`…), nunca colores fijos, salvo los de franja A/B/C definidos en 2.2.
6. El frontend no tiene pruebas automáticas. Verificar con `npx tsc --noEmit` y `npm run build`. Las funciones puras (`buildAgentInput`, validaciones, nivel por PCI, comparación de consultas, armado del PDF) van en `services/agent/`, sin React, para poder probarlas después.
7. **Guardado:** el proyecto se guarda con `API.put('/projects/:id')`, como las muestras, pero `App.tsx` copia campo por campo (`mapProjectToMongoPayload`, `mapMongoProjectToProject`) y `saveProjectToStorage` ignora un guardado si hay otro en curso. Hay que añadir los campos nuevos a esos mapeos y guardar solo en los momentos de 3.1 (sección 6.3). Sin esto, la configuración se pierde al recargar.

## Orden sugerido

1. Tipos y catálogo (`types.ts`, `services/agent/catalog.ts`).
2. Funciones puras: nivel por PCI, validaciones, `buildAgentInput`, `compareRuns`, lectura de `analisis.json`.
3. Navegación: paso 5 en `AppHeader.tsx`, caso `AGENT` en `App.tsx`, botón en Resultados.
4. Guardado: mapeos de `App.tsx` y, si se usa el backend de GitHub, `Project.js` y `server.js` (sección 6).
5. Modo Configurar: bloque por bloque (3.3 a 3.7) y barra de acción (3.8).
6. Cliente del servicio y modo sin servicio.
7. Modo Resultado: resumen, parámetros de la consulta, ajuste rápido, comparación e historial (3.9, 3.10) e informe y PDF (sección 4).
8. Traducciones, tema oscuro y vista móvil.

## Resultado esperado

Con un proyecto de la app, el inspector abre «5. Recomendaciones» desde el menú o desde el botón de Resultados, completa los bloques y pulsa *Generar recomendaciones*. El servicio (`uvicorn servicio.main:app --port 8000`) responde y la pantalla pasa al modo Resultado, con el resumen y el informe completo. *Editar parámetros* vuelve a la configuración; al volver a generar se ve qué cambió respecto a la consulta anterior. Sin servicio, descarga `entrada.json`, ejecuta `python -m agente todo entrada.json -d salida` e importa `analisis.json` y `narrativa.json`.

# 1 Lugar en la app y flujo

```
DASHBOARD → 1 Detalles (SETUP) → 2 Datos de Fallas (DATA_ENTRY) → 3 Cálculo (CALCULATION_DETAILS)
          → 4 Resultados (RESULTS) → 5 Recomendaciones (AGENT)          ← nuevo

AGENT:   Configurar ──[Generar recomendaciones]──> Consultando ──> Resultado
              ^                                          | error: vuelve a Configurar con el aviso
              └──────────[Editar parámetros]─────────────┘   (el resultado anterior se conserva)

Datos:  Project + section.context + project.agentConfig ──buildAgentInput()──> entrada.json
        entrada.json ──POST {VITE_AGENT_URL}/api/recommendations──> { analisis, narrativa }   (con servicio)
        entrada.json ──descargar──> python -m agente todo ──> importar analisis + narrativa      (sin servicio)
        { analisis, narrativa } ──> AgentRun en project.agentRuns ──> resumen + informe en la web ──> PDF (jsPDF)
```




Figura 1a. Menú lateral (`AppHeader.tsx`): el paso 5 se añade a `STEP_DEFINITIONS`.




Figura 1b. Resultados: botón primario junto a «Exportar» (`ResultsDisplay.tsx`, cabecera de la tarjeta).

- **La app no tiene barra de pasos horizontal** (`components/ProjectStepper.tsx` está vacío): los pasos están en el menú lateral de `AppHeader.tsx`. Añadir `{ view: 'AGENT', labelKey: 'stepper.step5', icon: AgentIcon }` a `STEP_DEFINITIONS` y `'AGENT'` al final de `VIEW_ORDER` (en `AppHeader.tsx` y en `App.tsx`). Textos: «5. Recomendaciones» / «5. Recommendations». El icono sigue el patrón de los demás (SVG de 20 px, `fill="currentColor"`).
- `App.tsx`: caso `'AGENT'` en el `switch` de `renderContent` (después de `RESULTS`); en `handleNavigate` la condición `view === 'CALCULATION_DETAILS' || view === 'RESULTS'` pasa a incluir `'AGENT'` (necesita el PCI calculado); en la carga del proyecto, `needsResultsCalc` también cuando `targetView === 'AGENT'`.
- **Entrada desde Resultados:** botón primario *Ir a Recomendaciones* (prop `onOpenAgent` que llama a `onNavigate('AGENT')`). Si algún tramo no tiene PCI, se deshabilita con un `InfoTip` que lo explica.
- **Última vista:** como el resto de pasos, si el usuario sale estando en Recomendaciones, el proyecto se reabre ahí (`lastView = 'AGENT'`; en el backend de GitHub `lastView` es un `String` libre, no hace falta cambiarlo).
- Si el PCI de algún tramo falta, la pantalla muestra un aviso bloqueante con botón *Ir a Cálculo* en lugar de los bloques.
- Si el proyecto cambia después de generar (se edita una muestra), el modo Resultado muestra el aviso *«Los datos cambiaron desde esta consulta»* (la huella de la entrada ya no coincide) con el botón *Volver a consultar*.

## 1.1 Tipos nuevos en `types.ts`

```ts
export type ProjectView = 'SETUP' | 'DATA_ENTRY' | 'CALCULATION_DETAILS' | 'RESULTS' | 'AGENT';
// MongoProject.lastView (misma unión escrita a mano en types.ts): añadir 'AGENT'

export type RoadType = 'urbana' | 'rural';
export type TrafficClass = 'bajo' | 'medio' | 'alto';
export type Drainage = 'bueno' | 'regular' | 'malo';

export interface SectionContext {          // Section.context?: SectionContext
  roadType: RoadType | null;
  curbHeightCm: number | null;              // solo urbana
  trafficInput: 'esal' | 'clase';           // pestaña elegida
  esalMillions: number | null;
  trafficClass: TrafficClass | null;
  aadt: number | null;                      // TPDA, informativo
  yearBuilt: number | null;
  lastInterventionYear: number | null;
  lastInterventionType: string | null;      // ID del catálogo u 'otra'
  drainage: Drainage | null;
  functionalClass: string | null;
  notes: string | null;
}

export interface UnitCost { valor: number; unidad: 'm2' | 'm' | 'global'; fuente?: string; }

export interface AgentConfig {              // Project.agentConfig?: AgentConfig (borrador vigente)
  availableTreatments: string[];            // IDs del catálogo
  unitCosts: Record<string, UnitCost>;      // solo los que el inspector cambió
  currency: string;                         // 'USD'
  budget: number | null;                    // null = sin tope
  priorityMode: 'costo_beneficio' | 'peor_primero';
  contingencyPct: number;                   // va a parametros.imprevistos_pct
  pciThresholds?: Partial<Record<'A1' | 'A2' | 'B1' | 'B2' | 'C1', number>>;
  parameters?: Record<string, unknown>;    // avanzados, misma clave que parametros_defecto.json
  updatedAt: string;
}

export interface AgentRun {                 // Project.agentRuns?: AgentRun[] (máx. 5, el más nuevo primero; ~65 kB c/u)
  id: string;
  number: number;                           // 1, 2, 3… consecutivo por proyecto (se muestra como «v3»)
  createdAt: string;
  inputHash: string;                        // sha256 de entrada.json canónico
  config: AgentConfig;                      // parámetros usados (para «Restaurar parámetros» y los chips)
  contexts: Record<string, SectionContext | null>;   // contexto de cada tramo (clave = Section.id) en ese momento
  input: unknown;                           // entrada.json
  analysis: unknown;                        // analisis.json
  narrative: unknown;                       // narrativa.json (textos del informe)
  narrativeSource: 'modelo' | 'automatica' | 'importado';
  agentVersion: string;                     // "version" de la respuesta
  status: 'listo' | 'error';
  error?: string;
}
```

Los datos nuevos son opcionales: los proyectos guardados antes de este cambio deben abrir sin migración. `Section.context` viaja dentro de `Project.sections`, que la app ya copia completo.

# 2 Sistema visual

La sección usa el mismo lenguaje que Resultados: una `Card` grande con la cabecera de título y botones, y dentro, bloques con borde. No se introduce ningún estilo nuevo.

## 2.1 Tokens de la app (ya definidos en `index.html`)

| Uso en la sección | Clase Tailwind | Valor claro |
|---|---|---|
| Fondo de la vista | `bg-bg-base` | #E6EBE0 |
| Tarjeta principal y bloques | `bg-bg-surface` + `Card` | #F7F9F4 |
| Acción principal, estado completo, bloque abierto | `bg-primary`, `ring-primary`, `border-primary` | #36C8C8 |
| Acción secundaria | `Button variant="secondary"` | #9BC1BC |
| Estado bloqueante, crítico | `text-danger`, `bg-danger/10` | #ED6A5A |
| Estado «se asumirá» (ámbar) | `bg-amber-100 text-amber-800 border-amber-300` · oscuro: `dark:bg-amber-900/30 dark:text-amber-200` | — |
| Texto secundario | `text-text-muted` | #6B7280 |
| Bordes | `border-border-base` / `border-border-muted` | #D1D5DB / #E5E7EB |
| Cifras | `font-mono tabular-nums` | — |

## 2.2 Colores de franja (mismos del informe)

A Mantenimiento preventivo `#2e7d32` · B Rehabilitación menor y mayor `#a35f00` · C Reconstrucción `#b3261e`. Siempre con el texto del nivel al lado, nunca solo color.

## 2.3 Qué componente de la app reutilizar para cada pieza

| Pieza de la sección | Reutilizar de la app |
|---|---|
| Título y botones de la cabecera | Cabecera de `ResultsDisplay`: `h2 text-2xl font-bold text-primary`, nombre del proyecto en `text-text-muted`, botones en `flex flex-wrap gap-2` |
| Bloque expandible | Patrón de `SectionRow` (`isExpanded`, `ChevronDownIcon` que gira, contenido con `animate-simple-fade-in`) dentro de `border border-border-muted rounded-xl bg-bg-surface`. Componente propio `AccordionBlock` |
| Tablas | `overflow-x-auto border border-border-muted rounded-lg`; `thead` con `bg-secondary/20 text-secondary-dark`; filas `odd:bg-secondary/5` y `hover:bg-primary/5` |
| Campos | `Input` con `label` e `info` (`InfoTip`); `select` nativo con las clases de `ProjectForm`; casillas `h-5 w-5 rounded text-primary focus:ring-primary` |
| Aviso informativo | `bg-secondary/10 p-4 rounded-lg border border-secondary/20` (como «parámetros de muestreo» en `ProjectForm`) |
| Ventanas (ver JSON, copiar contexto, importar) | `Modal` de `components/ui`, con los botones al pie como en `ExportModal` |
| Indicadores del resumen | `Card` con borde y cifra grande; el color del PCI usa `getPCIRating` de `constants.ts` como en Resultados |

## 2.4 Estados de cada bloque

| Estado | Señal en el encabezado del bloque | Regla |
|---|---|---|
| Completo | ● círculo turquesa con ✓ | Sin faltantes |
| Se asumirán datos | ● círculo ámbar con ! | Faltan datos opcionales: deja generar; el agente asume el caso conservador y lo declara |
| Bloquea | ● círculo terracota con × | Error que impide generar: el botón *Generar* queda deshabilitado y la barra de acción dice por qué |
| Cambiado | punto turquesa pequeño y texto «cambiado» | Solo en modo Editar: el valor difiere del de la consulta mostrada (3.10) |

## 2.5 Composición

- **Escritorio (≥ 1024 px):** dentro de `main` (`container max-w-7xl`), una `Card` con cabecera y, debajo, los cinco bloques en columna única a todo el ancho. Una columna lateral no cabe: el contenido es de tablas y casillas. La **barra de acción** queda fija al pie del área de desplazamiento (`sticky bottom-0` dentro de `main`, que es el contenedor con `overflow-y-auto`).
- **Cabecera de la app:** se oculta al bajar y reaparece al subir (ya implementado en `AppHeader`); no hace falta tratarlo.
- **Móvil (< 768 px):** los bloques se apilan igual; los encabezados miden al menos 44 px; las tablas pasan a lista de tarjetas; la barra de acción muestra solo el botón principal y un contador de avisos (sección 8).
- El borrador se guarda en los momentos de 3.1; no en cada tecla.

# 3 La pantalla «Recomendaciones»

## 3.1 Estructura y estados

Una sola pantalla con una `Card` principal. Tiene tres estados que cambian el contenido bajo la cabecera; no hay navegación interna entre páginas.

| Estado | Cuándo | Qué se ve |
|---|---|---|
| **Configurar** | Es el estado inicial si el proyecto no tiene consultas; también tras *Editar parámetros* y tras un error | Los cinco bloques (3.3 a 3.7) y la barra de acción (3.8) |
| **Consultando** | Entre *Generar recomendaciones* y la respuesta | Los bloques quedan visibles y bloqueados (`pointer-events-none opacity-70`); la barra de acción muestra el progreso y *Cancelar* |
| **Resultado** | Con una consulta lista (recién generada o la más reciente guardada) | Parámetros de la consulta, resumen e informe, comparación e historial (3.9) |

Al abrir la sección: sin consultas guardadas → Configurar; con consultas → Resultado de la más reciente (con aviso si los datos cambiaron). *Editar parámetros* lleva de Resultado a Configurar y *Ver resultado* regresa sin generar (3.10).

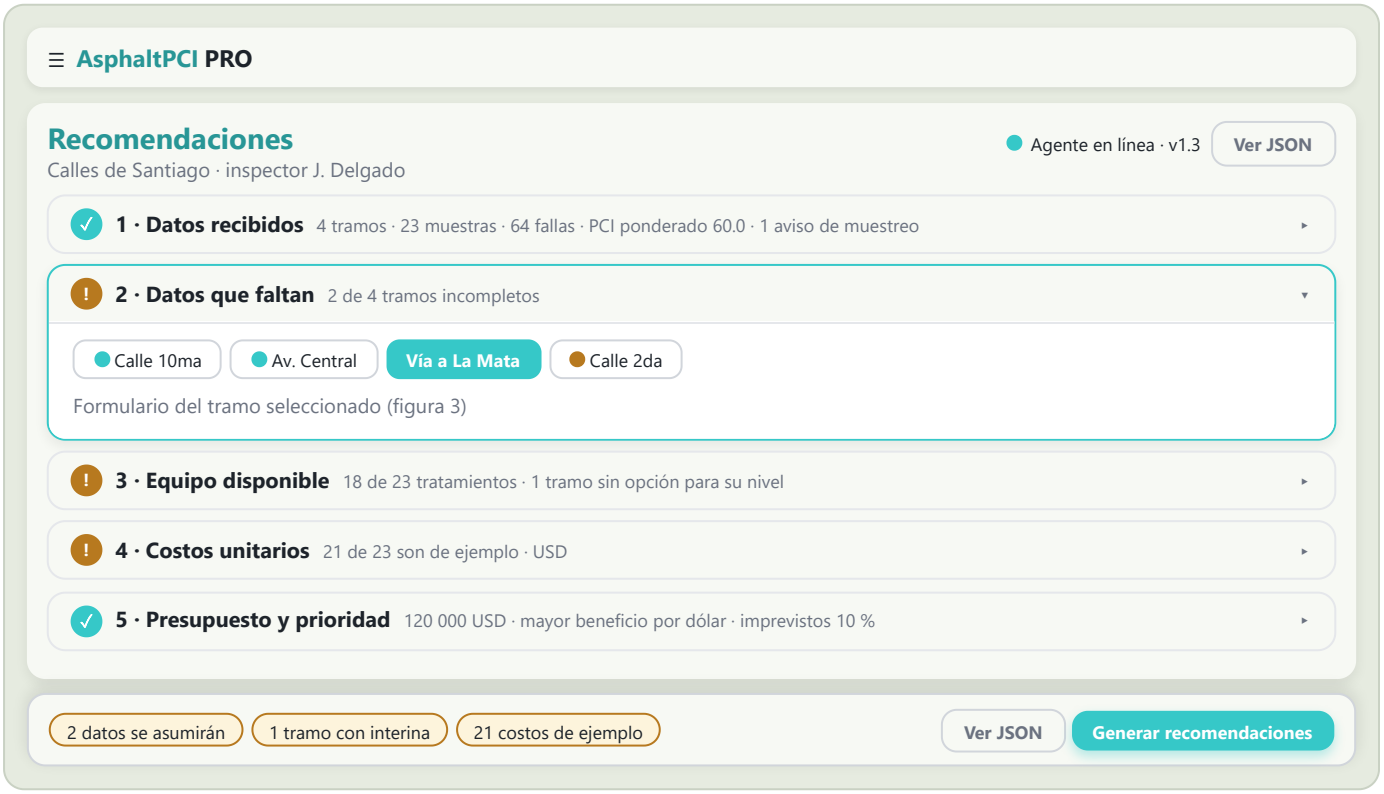



Figura 2. Estado Configurar. Cada bloque cerrado resume su contenido en una línea; la barra de acción queda fija al pie.

- **Bloques:** acordeón; se pueden abrir varios a la vez. Al entrar por primera vez, abierto el bloque 2 y cerrados los demás; si algún bloque está en rojo, se abre ese. Botón *Expandir todo / Contraer todo* en la cabecera de la tarjeta.
- **Línea de resumen** de cada bloque cerrado: dato clave actual, para revisar sin abrir. Es función pura de la configuración (`summarizeBlock(n, ctx)`).
- **Saltos:** los contadores de la barra de acción y los chips de 3.9 abren el bloque correspondiente y hacen `scrollIntoView`, con el foco en el título del bloque.
- **Cuándo se guarda el borrador** (`project.agentConfig` y `section.context`): al cerrar un bloque, al cambiar de tramo en el bloque 2, al pulsar *Generar*, al recibir el resultado y al salir de la vista. **No en cada tecla**: `saveProjectToStorage` ignora un guardado si hay otro en curso, así que el panel usa una cola (si hay uno en curso, marca «pendiente» y repite al terminar). El estado vivo está en `useState` del `AgentPanel`; el proyecto solo recibe copias.
- **Valores iniciales:** si el proyecto no tiene `agentConfig`, se toman las preferencias del usuario (7.1) y, si no hay, el catálogo y los valores por defecto del agente.

## 3.2 Cabecera y estado del agente

- Título «Recomendaciones» y nombre del proyecto, como en Resultados. A la derecha, el estado del agente y *Ver JSON*.

- Al abrir la sección se hace `GET {VITE_AGENT_URL}/` con 3 s de espera. Con respuesta: «Agente en línea · v{version}» y, si `redactor` es `automatico`, el texto «textos automáticos». Sin respuesta: «Sin servicio» con *Reintentar* y el botón principal cambia a las opciones de 3.8.
- Los modos Configurar y Resultado comparten la cabecera.

## 3.3 Bloque 1 · Datos recibidos (solo lectura)

Comprueba que el cálculo del PCI esté completo y muestra el nivel que le toca a cada tramo con los umbrales vigentes (que se pueden cambiar en el bloque 5). Cerrado por defecto salvo que tenga un aviso en rojo.

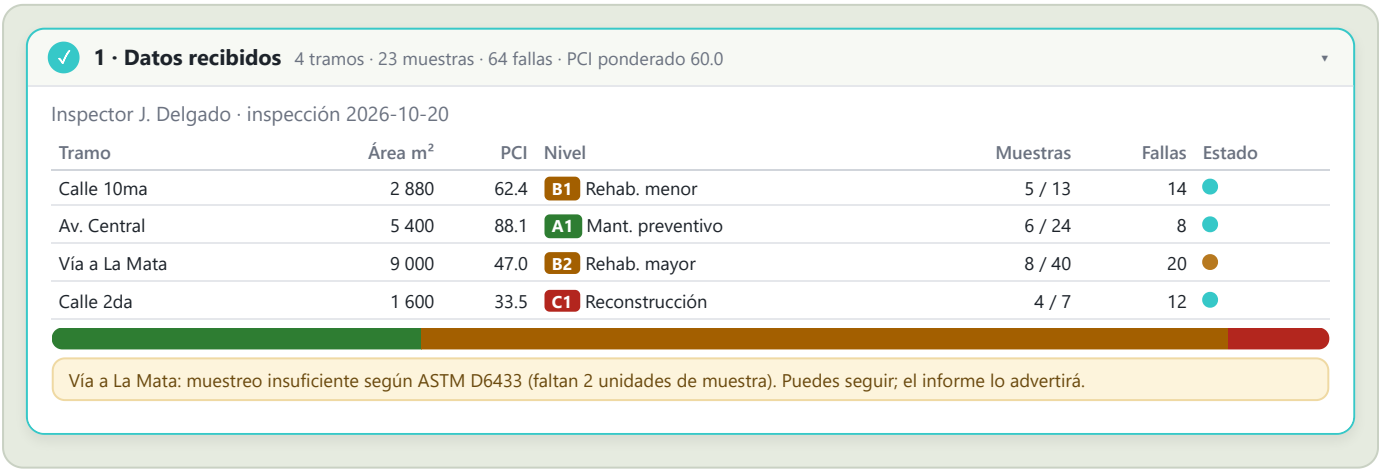



Figura 3a. Bloque 1 abierto. La barra de color es el área por franja (A, B, C).

| Comprobación | Nivel | Mensaje |
|---|---|---|
| Tramo sin `pci` | Bloquea | «Calcula el PCI de {tramo} antes de generar recomendaciones» (con botón *Ir a Cálculo*) |
| Tramo sin muestras, o muestra con área ≤ 0 | Bloquea | «{tramo}: muestra {id} sin área» |
| Pavimento distinto de flexible | Bloquea | «El agente solo cubre pavimento flexible» |
| `samplesSufficient === false` | Aviso | Muestreo insuficiente (ver figura) |
| Muestra sin fallas | Información | «La muestra {id} no tiene fallas registradas» (puede ser correcto) |
| \|PCI del tramo − promedio ponderado de sus muestras\| > 5 | Aviso | «El PCI del tramo difiere del promedio de sus muestras» |

Nivel por PCI (función pura `levelForPci(pci, thresholds)`): A1 ≥ 85, A2 ≥ 71, B1 ≥ 56, B2 ≥ 41, C1 ≥ 26, si no C2. Nombres: A = Mantenimiento preventivo, B1 = Rehabilitación menor, B2 = Rehabilitación mayor, C = Reconstrucción.

## 3.4 Bloque 2 · Datos que faltan (contexto de cada tramo)

Lo que la evaluación PCI no tiene y que el agente necesita para completar la matriz: tipo de vía, bordillo, tránsito, edad del pavimento, drenaje. Una pestaña por tramo (con su punto de estado) y un formulario debajo. Botón *Copiar este contexto a…* (modal con casillas: todos o algunos tramos; copia todo menos las observaciones). La lista «Se asumirá» está al pie del bloque.




Figura 3b. Bloque 2. Los campos vacíos que el agente asumirá se marcan en ámbar con el supuesto exacto.

| Campo | Control | Validación | Si queda vacío | Para qué lo usa el agente |
|---|---|---|---|---|
| Tipo de vía | Control segmentado de 2 opciones | — | Ámbar: «se asumirá urbana con bordillo» | Rural quita la regla del bordillo y agrega conformación de hombros |
| Altura libre de bordillo (cm) | Número, paso 0.5, con dibujo del corte calzada-bordillo en un `InfoTip` | 0–40; solo visible si urbana | Ámbar: «se asumirá < 7.6 cm: se fresará en vez de sobrecapa» | ≥ 7.6 cm (3 in) permite sobrecapa; menor obliga a fresar (MS-16, Fig. 1.4, Tabla 1.1) |
| Tránsito | Pestañas *ESAL* (millones, número ≥ 0) o *Clase* (bajo, medio, alto) | ESAL 0–200 | Ámbar: «se tratará como medio» | < 0.3 M bajo, < 3 M medio, resto alto (MS-2); restringe chip seal, slurry, sand seal; pondera el beneficio |
| TPDA | Número entero | ≥ 0 | Nada | Solo se copia al informe (no hay norma para convertirlo en clase) |
| Año de construcción (edad del pavimento) | Año | 1950 ≤ año ≤ año actual | Información: «la curva de deterioro usará el valor por defecto» | Edad para la curva de deterioro y la alerta de deterioro acelerado |
| Última intervención | Año + lista del catálogo u «otra» | ≥ año de construcción | Nada | Si existe, la edad se cuenta desde aquí |
| Drenaje | Radios: bueno, regular, malo | — | Nada | Malo fuerza reconstrucción en C1 y genera alerta |
| Clasificación funcional | Lista: arterial, colectora, local, rural primaria, rural secundaria | — | Nada | Informativo |
| Observaciones | Texto multilínea | ≤ 500 caracteres | Nada | Informativo |

El estado del bloque es ámbar si algún tramo tiene tipo de vía, bordillo (urbana) o tránsito vacíos; **nunca bloquea**. El punto de cada pestaña refleja a su tramo. La línea de resumen dice cuántos tramos están incompletos.

## 3.5 Bloque 3 · Equipo disponible (calibración del inspector)

El inspector marca qué puede ejecutar con su cuadrilla o contratista. El agente nunca recomienda algo sin marcar sin avisarlo: toma la siguiente opción de su lista y, si ninguna sirve, propone una solución interina marcada como crítica.

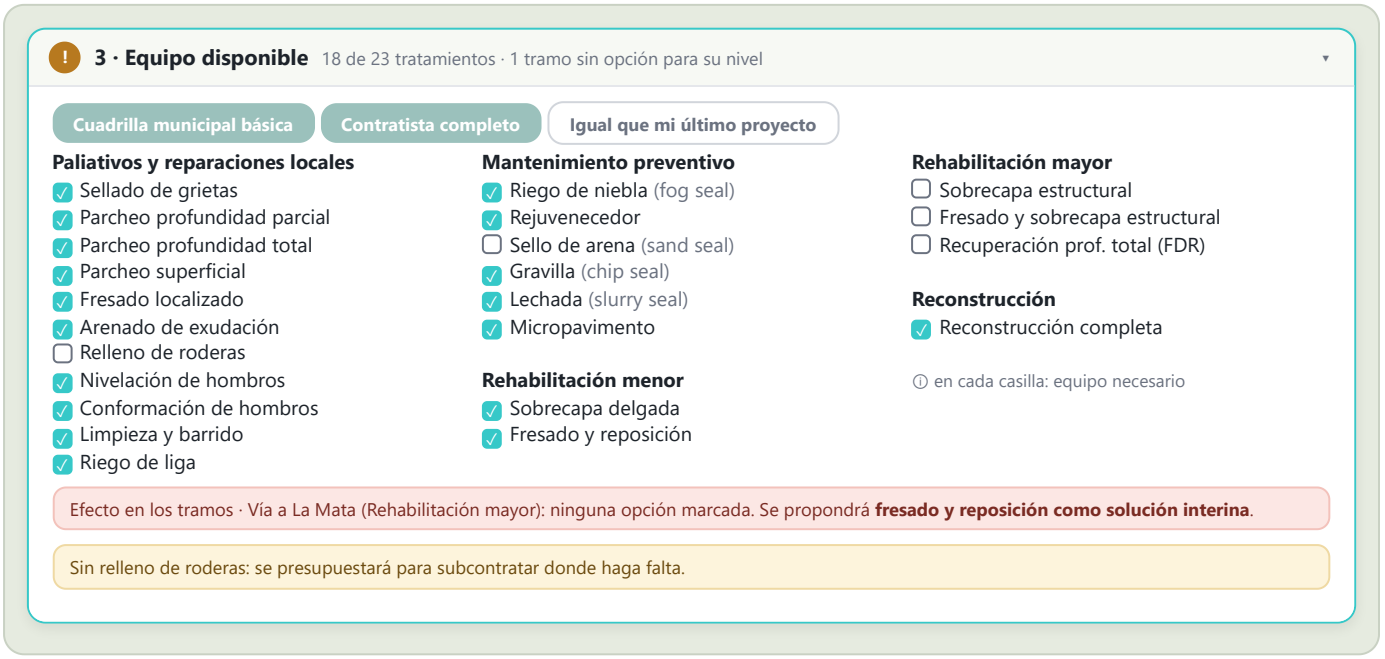



Figura 4. Bloque 3. El efecto de cada casilla se recalcula al instante y se muestra al pie del bloque.

- Cuatro grupos de casillas (los de la figura) con el nombre en español y el inglés en `text-text-muted` pequeño. Un `InfoTip` por casilla con la lista `equipo` del catálogo.
- **Atajos:** *Cuadrilla municipal básica* = todos los paliativos + fog seal + slurry seal; *Contratista completo* = todo; *Igual que mi último proyecto* = preferencia guardada del usuario (7.1).
- **Efecto en los tramos** (función pura): para cada tramo, según su nivel, ver si alguna opción del nivel está marcada. Opciones por nivel: A1/A2 cualquier tratamiento preventivo o el sellado; B1 `thin_overlay`, `mill_and_replace`; B2 `sobrecapa_estructural`, `fresado_sobrecapa_estructural`, `fdr`; C1 `reconstruccion`, `fdr`, `sobrecapa_estructural`, `fresado_sobrecapa_estructural`; C2 `reconstruccion`, `fdr`. Si no hay ninguna: aviso rojo con la interina (B1: sin interina; B2: `mill_and_replace` o `thin_overlay`; el resto, tomarlo de `niveles[].interinas` del catálogo). El estado del bloque es ámbar si algún tramo queda sin opción; la fuente de verdad son `niveles[].opciones` e `interinas` del catálogo.
- Bloquea solo si no hay ninguna casilla marcada.

## 3.6 Bloque 4 · Costos unitarios

Cargados por defecto con costos estándar de ejemplo. El inspector los puede calibrar uno a uno, importar la tabla de su empresa o dejarlos como están (el informe los marca como «ejemplo»).




Figura 5. Bloque 4.

- Filas: las actividades marcadas en el bloque 3 más todos los paliativos (siempre se presupuestan, aunque haya que subcontratarlos). Orden: paliativos, preventivo, rehabilitación, reconstrucción.
- *Propio*: número ≥ 0 con 2 decimales; vacío = usa el de ejemplo. Al escribir, la etiqueta pasa de EJEMPLO a CALIBRADO. *Fuente*: texto libre de 80 caracteres.
- La unidad es la del catálogo y no se edita. Si un CSV trae otra unidad para un ID, esa fila se rechaza en la vista previa (modal).
- **Importar CSV** sin librerías: separador coma o punto y coma, decimales con punto o coma, cabecera obligatoria, UTF-8. Excel queda fuera (guardar como CSV).
- El banner ámbar cuenta los costos de ejemplo que se usarán. No bloquea.
- En la entrada solo van los costos con *Propio* lleno, en `calibracion.costos_unitarios`.

## 3.7 Bloque 5 · Presupuesto y forma de priorizar

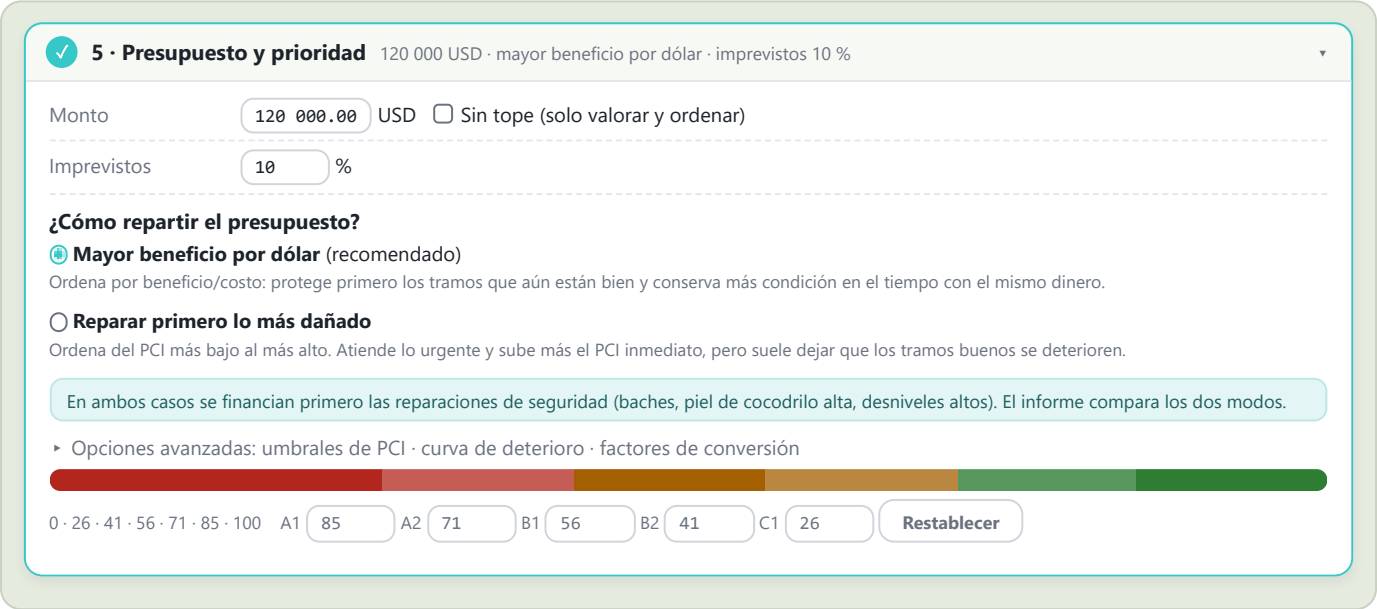



Figura 6. Bloque 5, con las opciones avanzadas desplegadas (cerradas por defecto).

| Campo | Regla | Va a la entrada como |
|---|---|---|
| Monto | Número > 0 con separador de miles; obligatorio salvo «Sin tope» | `calibracion.presupuesto` (o `null`) |
| Moneda | Fija «USD» en esta versión (texto, no selector) | `calibracion.moneda` |
| Imprevistos | 0–30 %, por defecto 10 | `calibracion.parametros.imprevistos_pct = {valor}` |
| Modo | Radio; por defecto `costo_beneficio` | `calibracion.modo_priorizacion` |
| Umbrales (avanzado) | Enteros, estrictamente decrecientes: 100 > A1 > A2 > B1 > B2 > C1 > 0. Barra 0–100 en vivo; botón *Restablecer*. Si no se cumplen: bloquea con el motivo | `calibracion.umbrales_pci` (solo si cambian) |
| Curva de deterioro (avanzado) | β 1.0–3.0 (defecto 1.5); años hasta el mínimo 5–40 (defecto 20); PCI mínimo 20–60 (defecto 40) | `parametros.curva_deterioro`, `parametros.pci_minimo_aceptable` |
| Factores (avanzado) | Área por bache L/M/H, sobreancho de parcheo, metros de grieta por m² de bloque, umbral estructural %; cada uno con su origen (norma o supuesto) al lado | `parametros.*` (solo si cambian) |

## 3.8 Barra de acción: revisar y generar

Fija al pie del área de desplazamiento en el modo Configurar. A la izquierda, los avisos que el agente declarará (cada chip abre su bloque); a la derecha, las acciones.

| Elemento | Contenido y comportamiento |
|---|---|
| Chips de aviso (ámbar) | «{n} datos se asumirán» → bloque 2 · «{n} tramos con solución interina» → bloque 3 · «{n} costos de ejemplo» → bloque 4 · «muestreo insuficiente en {n} tramos» → bloque 1 |
| Chip de bloqueo (rojo) | «No se puede generar: {motivo}» → bloque que lo causa. Mientras exista, *Generar* está deshabilitado |
| *Ver JSON* | `Modal` con el `entrada.json` formateado (`font-mono`) y botón copiar / descargar |
| ***Generar recomendaciones*** | Valida, guarda el borrador y llama al agente (ver abajo) |

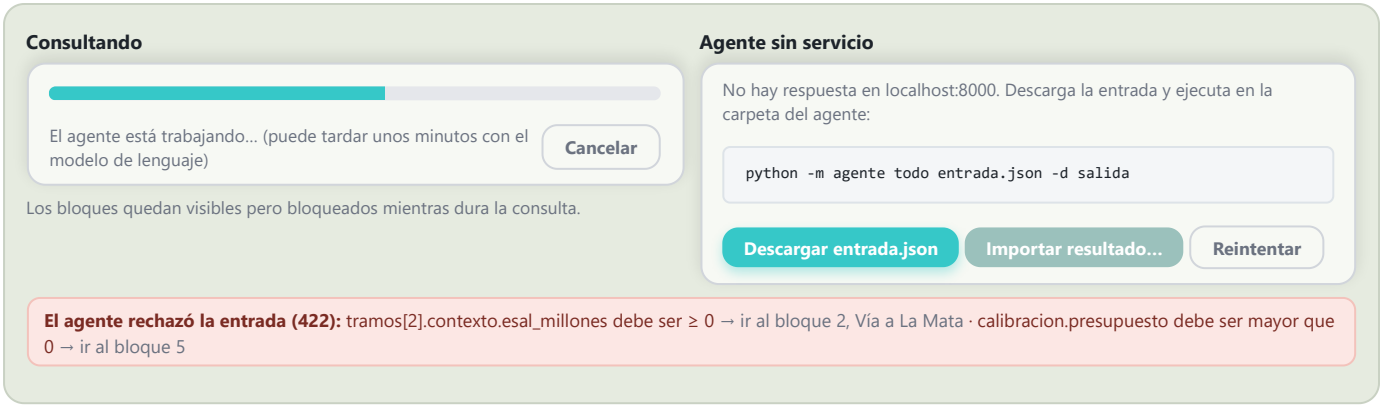



Figura 7. Estados de la barra de acción y aviso de error.

- **Generar:** `POST {VITE_AGENT_URL}/api/recommendations` con la entrada. La llamada es síncrona y puede tardar varios minutos si el redactor es un modelo de lenguaje (espera máxima 6 min, *Cancelar* con `AbortController`). Con **200** se crea el `AgentRun` (número = el mayor + 1), se guarda el proyecto, y la pantalla pasa al estado Resultado con el desplazamiento arriba. Con **422** se vuelve a Configurar con el aviso rojo de la figura; cada error se enlaza al bloque según su ruta (`tramos[i].contexto.*` → 2 con el tramo `i`; `tratamientos_disponibles` → 3; `costos_unitarios` → 4; `presupuesto`, `modo_priorizacion`, `umbrales_pci`, `parametros` → 5). Con error de red o 500: mensaje con *Reintentar* y las opciones de modo sin servicio.
- **Sin servicio:** *Importar resultado…* abre un `Modal` que pide `analisis.json` y `narrativa.json` (los dos que deja `python -m agente todo`), los valida por su forma (campos obligatorios de los esquemas) y comprueba que `analisis.proyecto.huella_entrada` coincida con la huella de la entrada actual; si no coincide, avisa «el resultado es de otros datos» y deja continuar bajo confirmación. El `AgentRun` queda con `narrativeSource: 'importado'`.
- Cada generación o importación crea un `AgentRun` (máximo 5; al pasar, se borra el más antiguo con confirmación).

## 3.9 Estado Resultado: resumen e informe en pantalla

Con una consulta lista, el contenido de la tarjeta cambia a: (1) **los parámetros de la consulta** como chips, con *Editar parámetros* y un *Ajuste rápido*; (2) el **resumen** y el **informe** en dos pestañas; (3) la comparación con la consulta anterior y el historial. La web no recalcula nada: todas las cifras salen de `analisis` y todos los textos de `narrativa`.

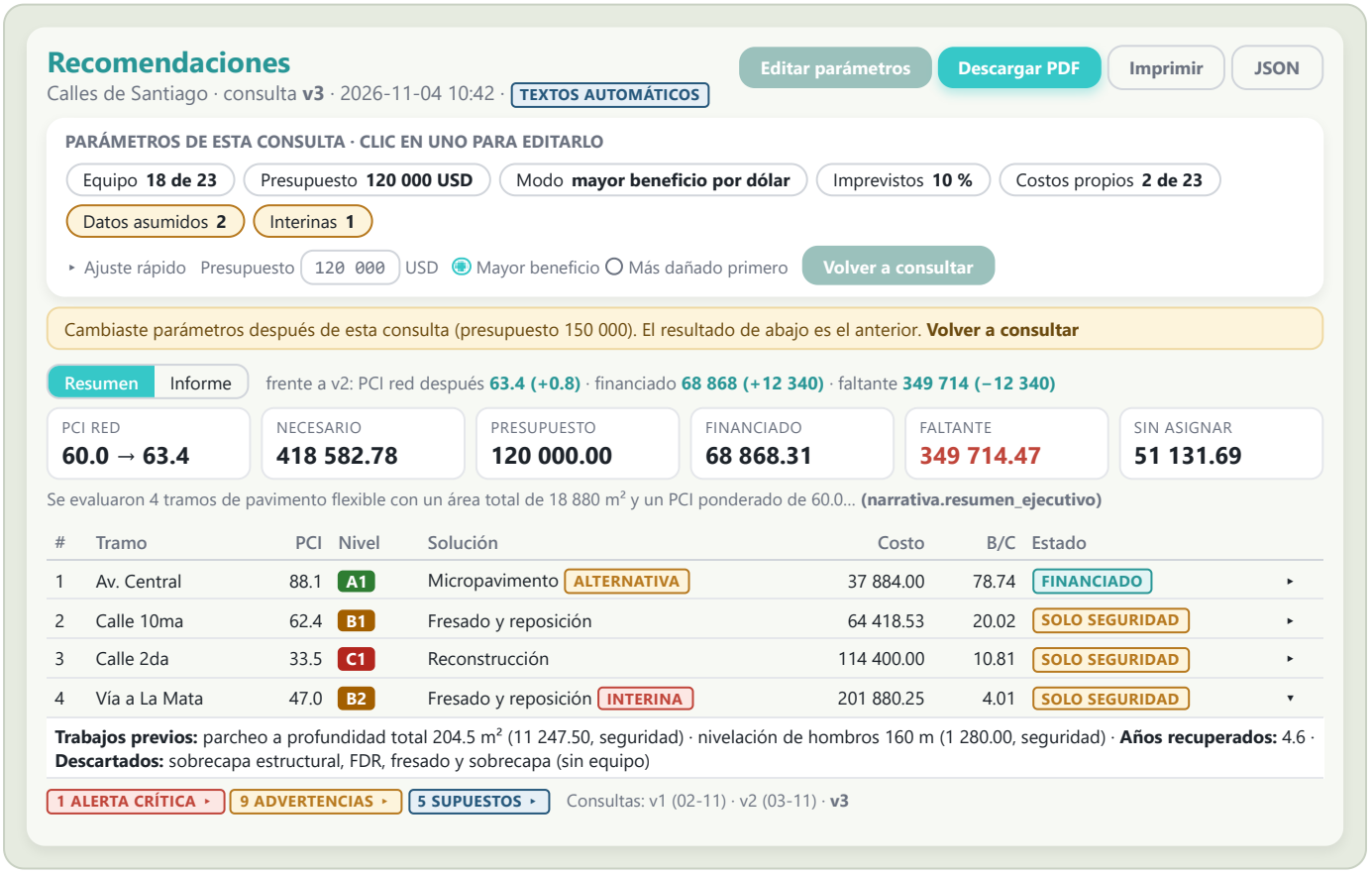



Figura 8. Estado Resultado, pestaña Resumen, con las cifras reales del ejemplo del agente. La franja ámbar «Cambiaste parámetros…» solo aparece si la configuración actual difiere de la consulta.

| Elemento | De dónde sale y cómo se comporta |
|---|---|
| Chips «Parámetros de esta consulta» | Se arman con `run.config` y `run.contexts` (no con el borrador actual): equipo (n de 23), presupuesto o «sin tope», modo, imprevistos, costos propios, datos asumidos (suma de `tramos[].contexto_usado.asumidos`), tramos con interina. Cada chip es un botón: abre el estado Configurar con ese bloque abierto (3.10) |
| Ajuste rápido | Plegado por defecto. Presupuesto y modo de priorizar, los dos parámetros que más se prueban. Escribe en el borrador y *Volver a consultar* genera de inmediato, sin pasar por Configurar. Usa las mismas validaciones del bloque 5 |
| Franja «Cambiaste parámetros» | Se muestra si la huella de `buildAgentInput(project, borrador)` es distinta de `run.inputHash` (por cambios en el borrador o en las muestras del proyecto) |
| Comparación con la consulta anterior | Función pura `compareRuns(nueva, anterior)` sobre `analisis.red` y `analisis.tramos`: diferencia de PCI de la red después, necesario, financiado, faltante y sin asignar, con color (verde mejora, terracota empeora); y la lista de tramos cuya solución o estado cambió («Vía a La Mata: sobrecapa → fresado y reposición»). Solo aparece si hay una consulta anterior |
| PCI red antes → después | `analisis.red.pci_ponderado`, `pci_ponderado_despues` |
| Necesario, financiado, faltante, sin asignar | `analisis.red.costo_total_necesario`, `costo_financiado`, `faltante`, `saldo_sin_asignar`; presupuesto en `analisis.calibracion_usada.presupuesto` |
| Texto bajo los indicadores | `narrativa.resumen_ejecutivo` |
| Filas en orden | `analisis.red.orden_ejecucion` → `tramos[id]`: `prioridad.posicion`, `pci`, `nivel_id`, `tratamiento.nombre` (o `nombre_ideal` si `seleccionado` es null), `tratamiento.interino`, `costos.total`, `relacion_bc`, `prioridad.estado` |

| Elemento | De dónde sale y cómo se comporta |
|---|---|
| Etiqueta «alternativa» / «interina» | `seleccionado ≠ ideal` / `interino` |
| Fila expandida | `trabajos_previos[]` (nombre, cantidad, unidad, costo, seguridad), `beneficio_detalle.anios_recuperados`, `tratamiento.descartados[]` (nombre, razón) |
| Estados | `financiado` turquesa, `solo_seguridad` ámbar, `diferido` terracota, `sin_presupuesto` gris |
| Contadores | `tramos[].alertas[]` + `alertas_globales[]` por `nivel`; `tramos[].supuestos[]` |
| Pestaña Informe | El documento completo (sección 4), dibujado con `AgentReport` |
| Botones | *Descargar PDF* con jsPDF (4.3). *Imprimir* usa `window.print()` con una hoja de estilo que oculta todo menos el informe. *JSON* baja la respuesta completa |
| Historial | Lista «Consultas: v1 · v2 · v3» al pie del resumen; al elegir una anterior se muestra en solo lectura con la franja «Estás viendo la v2» y dos botones: *Restaurar sus parámetros* (copia `run.config` y `run.contexts` al borrador y abre Configurar) y *Volver a la más reciente* |

## 3.10 Editar parámetros

*Editar parámetros* (y cualquier chip) lleva del estado Resultado al estado Configurar, **sin perder nada**: los bloques se abren con los valores de la consulta, listos para cambiarlos.

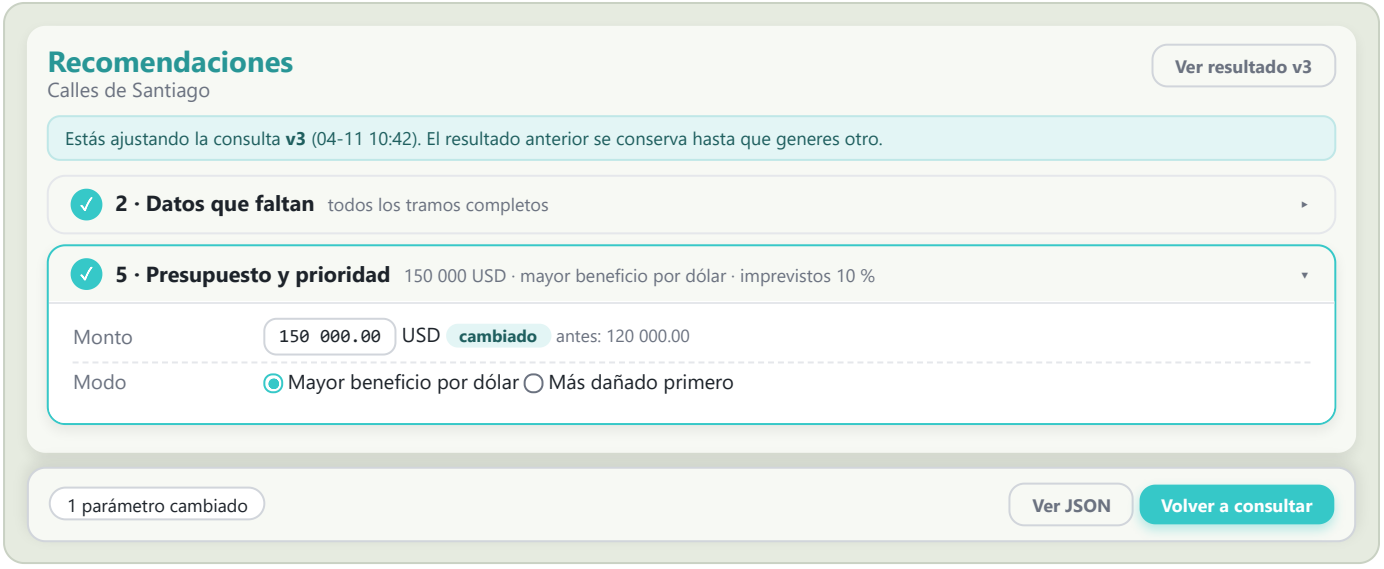



Figura 9. Estado Configurar al editar una consulta. Los campos modificados se marcan «cambiado» con el valor anterior.

- **Qué cambia respecto a la primera vez:** la cabecera trae *Ver resultado v{n}* (vuelve al Resultado sin generar) y un aviso azul; la barra de acción dice *Volver a consultar* y cuenta los parámetros cambiados.
- **«Cambiado»:** cada campo cuyo valor difiere de `run.config` o `run.contexts` lleva la etiqueta y el valor anterior en `text-text-muted`. Un bloque con algún campo cambiado lo indica en su línea de resumen.
- Si la huella de la entrada nueva es igual a la de la consulta, *Volver a consultar* sigue habilitado, con la nota «Sin cambios: las cifras serán idénticas; solo pueden variar los textos si el redactor es un modelo de lenguaje».
- Al terminar la nueva consulta se crea la v{n+1}; la anterior pasa al historial. Con 5 consultas, se pide confirmación para borrar la más antigua.
- Los chips del estado Resultado abren el mismo estado con el bloque correspondiente abierto y el foco en él. «Datos asumidos» → bloque 2; «Interinas» y «Equipo» → bloque 3; «Costos propios» → bloque 4; presupuesto, modo e imprevistos → bloque 5.

# 4 Informe en la web

El informe se dibuja con componentes React a partir de la respuesta del agente. Debe contener lo mismo que `docs/panel/informe-referencia.pdf` (generado por el agente en Python con el ejemplo), que sirve como modelo visual y de contenido. Componente raíz: `components/agent/report/AgentReport.tsx`, con props `{ project, run }`.

## 4.1 Secciones y su origen

| Sección | Contenido | Datos |
|---|---|---|
| Portada | Título «Informe de recomendaciones de mantenimiento y rehabilitación vial», proyecto, entidad, ubicación, inspector, fechas, código, versión del agente, origen de los textos, aviso de apoyo a la decisión | `analisis.proyecto` (`huella_entrada` [0:8] = código), `run.createdAt`, `run.agentVersion`, `run.narrativeSource` |
| 1 Resumen ejecutivo | 6 indicadores, texto, barra de área por nivel con tabla, caja de alertas críticas | `analisis.red`, `narrativa.resumen_ejecutivo`, `tramos[].nivel_id/area_m2` |
| 2 Alcance y metodología | Texto fijo (traducible) con los 6 pasos del agente, barra de niveles con los umbrales usados, explicación del beneficio, huella | `calibracion_usada.umbrales_pci`, `parametros.pci_minimo_aceptable`, `parametros.curva_deterioro` |
| 3 Datos y calibración | Tabla de tramos con contexto (asterisco en los asumidos), equipo disponible marcado ✓/✗, costos usados con etiqueta ejemplo/calibrado | `tramos[].contexto_usado` (incluye `asumidos[]`), `calibracion_usada.tratamientos_disponibles`, `calibracion_usada.costos` |
| 4 Ficha por tramo | Cabecera con color de franja y PCI; barra de PCI con marcas antes y después; solución recomendada y 5 datos (costo, costo/m², años recuperados, B/C, prioridad); diagnóstico con tabla de fallas; justificación; descartados; nota de equipo; trabajos previos; tabla de presupuesto con «Cálculo de la cantidad»; subtotal, imprevistos y total; si no se interviene; alertas; referencias | `tramos[i]` completo y `narrativa.tramos[tramo_id]` (`diagnostico`, `justificacion`, `trabajos_previos`, `consecuencia_si_se_difiere`, `nota_equipo`) |
| 5 Priorización | Texto; tabla con asignado y acumulado; filas de total, presupuesto, faltante y saldo sin asignar; tabla de comparación de modos | `narrativa.priorizacion`, `red.orden_ejecucion`, `red.comparacion_modos`. Asignado: total si `financiado`, `costo_interino_si_diferido` si `solo_seguridad`, 0 si no |
| 6 Alertas | Tabla ordenada crítica → advertencia → información | Texto de `narrativa.alertas[]` por `(codigo, tramo_id)`; si falta, `alerta.mensaje` |
| 7 Conclusiones | Tres listas | `narrativa.conclusiones`, `recomendaciones`, `limitaciones` |
| 8 Referencias | Remisiones usadas y bibliografía fija | Unión de `tramos[].referencias`; bibliografía en `services/agent/reportText.ts` (copiar de la sección 8 de la referencia) |
| 9 Revisión y firma | Declaración y tabla: elaborado por, revisado por, idoneidad JTIA n.º, firma, fecha, observaciones (en blanco) | `analisis.proyecto.inspector` |
| Anexos A, B, C | Fallas por unidad de muestra; glosario; parámetros con su origen y supuestos por tramo | `tramos[].muestras`, `narrativa.glosario`, `calibracion_usada.parametros`, `tramos[].supuestos` |

## 4.2 Pantalla e impresión

- En pantalla: hoja blanca de ancho A4 centrada sobre `bg-bg-base`, índice lateral fijo para saltar entre secciones y tramos (en móvil, menú desplegable).
- El informe usa siempre fondo blanco y texto oscuro, también en tema oscuro, porque es un documento.
- `@media print`: ocultar cabecera, menú, barra de parámetros y botones; `break-before: page` en cada sección numerada y cada ficha de tramo; `thead` repetido; `tr { break-inside: avoid }`; `print-color-adjust: exact`.
- Formato de números: separador de miles con espacio y 2 decimales para dinero (`1 234.56`), 1 decimal para PCI, `m²` para áreas.

## 4.3 PDF con jsPDF (`services/agent/agentReportPdf.ts`)

- Seguir el patrón de `generatePdfReport` en `services/exportService.ts` (`window.jspdf.jsPDF` + `doc.autoTable`), A4 vertical, márgenes de 16 mm, fuente Helvetica.
- Mismas secciones y en el mismo orden que 4.1: textos con `doc.splitTextToSize`, tablas con `autoTable` (encabezado azul `#0b3c5d` y texto blanco, filas alternas), salto de página antes de cada sección y de cada ficha de tramo.
- Barras de PCI y de niveles con `doc.rect` relleno en los colores de franja; marcas del PCI actual (relleno) y esperado (contorno).
- Pie en cada página: proyecto, «Informe {código}», «Página X de Y» (segunda pasada con `doc.getNumberOfPages()`).
- Nombre del archivo: `{proyecto}_recomendaciones_v{n}.pdf`.

# 5 Armado de `entrada.json`

Función pura `buildAgentInput(project: Project, config: AgentConfig): AgentInput` en `services/agent/buildAgentInput.ts`. El resultado debe validar contra `schemas/entrada.schema.json` (versión `"1.0"`, `additionalProperties: false`: no mandar campos de más).

| Entrada | Origen en la app | Nota |
|---|---|---|
| `proyecto.id / nombre / inspector / fecha_inspeccion` | `Project.id / name / inspector / inspectionDate` | Fecha en `YYYY-MM-DD` |
| `proyecto.descripcion`, `entidad` | `Project.descripcion`; `MongoUser.company` | Omitir si vacíos |
| `tramos[].id / nombre / longitud_m / ancho_m / area_m2 / pci` | `Section.id / name / length / width / area / pci` | |
| `tramos[].ubicacion` | `Section.location` | Omitir si vacío |
| `tramos[].total_unidades_muestra / muestreo_suficiente` | `Section.totalSampleUnits / samplesSufficient` | |
| `tramos[].contexto.*` | `Section.context` | `roadType→tipo_via`, `curbHeightCm→altura_bordillo_cm`, `esalMillions→esal_millones` (solo si pestaña ESAL), `trafficClass→clase_trafico` (solo si pestaña Clase), `aadt→tpda`, `yearBuilt→anio_construccion`, `lastInterventionYear→anio_ultima_intervencion`, `lastInterventionType→ultima_intervencion`, `drainage→drenaje`, `functionalClass→clasificacion_funcional`, `notes→observaciones`. Vacíos como `null` |
| `muestras[].id / area_m2 / adicional` | `Sample.id / area / isAdditional` | |
| `muestras[].pci` | `Sample.calculationDetails?.pci` | Omitir si no existe |
| `fallas[].tipo / severidad / cantidad` | `Distress.type / severity / quantity` | Una fila por `Distress`, con la severidad tal como la guarda la app; omitir cantidades ≤ 0 |
| `fallas[].valor_deducido` | `calculationDetails.summaries[]` con el mismo `type` y `severity` | Si hay varias filas del mismo tipo y severidad en la muestra, repartir el deducido en proporción a la cantidad, o mandarlo solo en la primera |
| `calibracion.tratamientos_disponibles` | `config.availableTreatments` | |
| `calibracion.costos_unitarios` | `config.unitCosts` | Solo los calibrados |
| `calibracion.moneda / presupuesto / modo_priorizacion` | `config.currency / budget / priorityMode` | |
| `calibracion.umbrales_pci` | `config.pciThresholds` | Omitir si no cambió |
| `calibracion.parametros` | `config.parameters` + `{ imprevistos_pct: { valor: contingencyPct } }` | |

**Huella:** `sha256` del JSON con claves ordenadas y sin espacios (`crypto.subtle.digest`); se guarda en el `AgentRun` y se compara con `analisis.proyecto.huella_entrada` al importar. Para que coincidan, serializar igual que Python: `json.dumps(entrada, sort_keys=True, separators=(',', ':'), ensure_ascii=False)`.

Ejemplo completo de entrada válida: `agente-recomendaciones/ejemplos/entrada_ejemplo.json`.

# 5.1 Conexión con el servicio del agente

| Llamada | Uso |
|---|---|
| `GET {VITE_AGENT_URL}/` | Comprobar si el agente está en línea (3 s de espera). Responde `{status, version, redactor}` |

| Llamada | Uso |
| --- | --- |
| `GET {VITE_AGENT_URL}/api/recommendations/catalogo` | Catálogo vigente: tratamientos (id, nombre, unidad, equipo, costo de ejemplo), fallas, niveles con `opciones` e `interinas`, parámetros. Se pide al abrir la sección; si falla, se usa la copia de `catalog.ts` |
| `POST {VITE_AGENT_URL}/api/recommendations` con `entrada.json` | Síncrona. **200** `{success: true, version, analisis, narrativa, narrativa_origen}` · **422** `{success: false, errores: [...]}` · 500 error del agente |

- `services/agent/agentClient.ts` con `fetch` (`baseURL = import.meta.env.VITE_AGENT_URL || 'http://localhost:8000'`; la copia local no tiene `axios`), espera de 6 min en el POST y cancelación con `AbortController` al salir de la vista o con *Cancelar*. No mandar el token de la app: el agente no lo usa.
- El agente corre en la máquina del inspector (usa Ollama local, como indica el README del repo). La web publicada en `https://asphaltpcipro.company` puede llamarlo en `http://localhost:8000`: el servicio ya responde con CORS para ese dominio y con `Access-Control-Allow-Private-Network`. Si el navegador lo bloquea, queda el modo sin servicio.
- Documentar `VITE_AGENT_URL` en `.env.example` y en el README.

# 6 Guardado y backend

Los datos nuevos (`section.context`, `project.agentConfig`, `project.agentRuns`) tienen que sobrevivir a guardar y recargar. Hay tres lugares donde se pueden perder; los dos primeros aplican a la copia local (puerto 5000) y a GitHub, el tercero solo a GitHub.

## 6.1 Mapeos de `App.tsx` (los dos casos)

`App.tsx` no pasa el proyecto completo al guardar: lo copia campo por campo, y un campo que no esté en la lista se pierde sin avisar.

```
// mapMongoProjectToProject (línea ~21): añadir
agentConfig: mongoProject.agentConfig,
agentRuns: mongoProject.agentRuns || [],

// mapProjectToMongoPayload (línea ~34): añadir
agentConfig: project.agentConfig,
agentRuns: project.agentRuns || [],

// types.ts: Project y MongoProject → agentConfig?: AgentConfig; agentRuns?: AgentRun[]
// Section → context?: SectionContext   (viaja dentro de sections, que ya se copia completo)

// saveProjectToStorage, rama PUT, objeto mergedProject (línea ~149): añadir
agentConfig: projectToSave.agentConfig,
agentRuns: projectToSave.agentRuns,
```

Sin la última línea, el `mergedProject` vuelve a la lista con los valores del servidor y la pantalla muestra datos viejos justo después de guardar.

## 6.2 Cuota del almacenamiento local (copia local, puerto 5000)

`services/localApi.ts` guarda todo en `localStorage` del navegador (unos 5 MB en total). `updateProject` acepta cualquier campo del cuerpo, así que no hace falta cambiarlo, pero cada `AgentRun` pesa unos 65 kB (análisis ~35 kB, narrativa ~25 kB, configuración y entrada el resto) y son hasta 5 por proyecto.

- Mantener como máximo 5 consultas por proyecto. Si el guardado falla con el 500 «No se pudo guardar en el almacenamiento local» (`QuotaExceededError`), el panel muestra el aviso y ofrece borrar la consulta más antigua y reintentar.
- Al guardar una consulta, quitar de `AgentRun` el `input` si ya se puede reconstruir con `buildAgentInput(project, run.config)`; se conserva `inputHash`. Es opcional: solo si la cuota resulta un problema en las pruebas.

## 6.3 Momentos de guardado

`saveProjectToStorage` descarta la petición si ya hay un guardado del mismo proyecto en curso (`savingProjectIdRef`). Por eso el panel no guarda en cada tecla. Reglas:

- Estado vivo del formulario en `useState` del `AgentPanel`; el proyecto solo recibe copias en los momentos de 3.1.
- Un `saveDraft()` con cola: si hay un guardado en curso, marca «pendiente» y repite al terminar con el estado más reciente.
- El guardado final al salir de la vista se hace antes de `onNavigate` (esperando la promesa), no solo en el `cleanup` del efecto.
- Un handler nuevo en `App.tsx`, `handleAgentSave(patch)`, mezcla `{ agentConfig, agentRuns, sections }` en `selectedProject` y llama a `saveProjectToStorage`, igual que `handleSampleUpdate`.
- Los cambios de `section.context` usan el mismo handler y reemplazan solo el tramo editado.

## 6.4 Backend de GitHub (solo si se usa ese servidor)

`server/models/Project.js` declara cada campo del proyecto y de la sección, y Mongoose descarta en silencio los que no estén declarados (`PUT /api/projects/:id` usa `findOneAndUpdate`). `lastView` ya es un `String` libre: `'AGENT'` se acepta sin cambios.

```
// SectionSchema: añadir
context: { type: mongoose.Schema.Types.Mixed },

// ProjectSchema: añadir
agentConfig: { type: mongoose.Schema.Types.Mixed },
agentRuns:   { type: [mongoose.Schema.Types.Mixed], default: [] },
```

`server/server.js` usa `express.json()`, cuyo límite por defecto es 100 kB: el proyecto lo supera en la segunda consulta y el guardado falla con 413. Cambiar a `express.json({ limit: '5mb' })`.

### 6.5 Reemplazo de `agent/` (tarea aparte, no de esta sección)

El `agent/` del repositorio de GitHub (LangGraph + Ollama) devuelve texto libre con cortes de PCI distintos de los de la tesis, y no calcula costos, trabajos previos ni prioridad. El paquete `agente-recomendaciones` lo reemplaza conservando FastAPI, Ollama y la ruta `POST /api/recommendations`, pero con respuesta estructurada: `uvicorn servicio.main:app --port 8000`. Ver `INTEGRACION.md` del paquete. La copia local no tiene `agent/`: para probar la sección, Dk lo arranca desde el paquete.

# 7 Catálogo para la interfaz (`services/agent/catalog.ts`)

Fuente principal: `GET /api/recommendations/catalogo`. Copia de respaldo en TypeScript (la misma estructura) para trabajar sin servicio: ID, nombre, nombre en inglés, grupo de interfaz, unidad, equipo, costo de ejemplo y opciones e interinas por nivel. Los nombres traducidos de la interfaz van a `translations` con clave `agent.treatments.{id}`.

| ID | Nombre | Grupo en el bloque 3 | Unidad | Ejemplo USD |
|---|---|---|---|---|
| `sellado_grietas` | Sellado y relleno de grietas | Paliativos | m | 2.50 |
| `parcheo_profundidad_parcial` | Parcheo a profundidad parcial | Paliativos | m² | 35.00 |
| `parcheo_profundidad_total` | Parcheo a profundidad total | Paliativos | m² | 60.00 |
| `parcheo_superficial` | Parcheo superficial / nivelación | Paliativos | m² | 25.00 |
| `fresado_local` | Fresado localizado y reposición | Paliativos | m² | 20.00 |
| `arenado_exudacion` | Arenado de la exudación | Paliativos | m² | 1.00 |
| `relleno_roderas` | Relleno de roderas | Paliativos | m² | 8.00 |
| `nivelacion_hombros` | Nivelación de hombros | Paliativos | m | 8.00 |
| `conformacion_hombros` | Conformación de hombros | Paliativos | m | 10.00 |
| `limpieza_superficie` | Barrido y limpieza | Paliativos | m² | 0.30 |
| `riego_liga` | Riego de liga | Paliativos | m² | 0.60 |
| `fog_seal` | Riego de niebla | Mantenimiento preventivo | m² | 1.20 |
| `rejuvenecedor` | Rejuvenecedor | Mantenimiento preventivo | m² | 1.50 |
| `sand_seal` | Sello de arena | Mantenimiento preventivo | m² | 2.00 |
| `chip_seal` | Tratamiento superficial con gravilla | Mantenimiento preventivo | m² | 4.50 |
| `slurry_seal` | Lechada asfáltica | Mantenimiento preventivo | m² | 4.00 |
| `micropavimento` | Micropavimento | Mantenimiento preventivo | m² | 6.00 |
| `thin_overlay` | Sobrecapa delgada (≤ 40 mm) | Rehabilitación menor | m² | 12.00 |
| `mill_and_replace` | Fresado y reposición | Rehabilitación menor | m² | 16.00 |
| `sobrecapa_estructural` | Sobrecapa estructural | Rehabilitación mayor | m² | 22.00 |
| `fresado_sobrecapa_estructural` | Fresado y sobrecapa estructural | Rehabilitación mayor | m² | 28.00 |
| `fdr` | Recuperación a profundidad total | Rehabilitación mayor | m² | 30.00 |
| `reconstruccion` | Reconstrucción completa | Reconstrucción | m² | 65.00 |

`monitoreo` no aparece en la interfaz: no requiere equipo. Los costos de ejemplo no tienen fuente; la interfaz siempre los marca como «ejemplo».

## 7.1 Preferencias del usuario

Clave `asphalt-pci-mapa.agent-defaults.v1` en `localStorage`, por usuario: `{ [userId]: { availableTreatments, unitCosts, priorityMode, contingencyPct } }`. Se escriben con *Guardar como mis costos* y al generar; se leen para proyectos nuevos y para el atajo *Igual que mi último proyecto*. Envolver lecturas y escrituras en `try/catch`.

# 8 Vista móvil y accesibilidad




Figura 10. Configurar en móvil (375 px).

- Los bloques se apilan; cada encabezado es un botón de 44 px de alto como mínimo y muestra solo el número, el título corto y el estado; la línea de resumen pasa debajo del título.
- Las pestañas de tramos se desplazan en horizontal; el tramo activo se centra.
- Las tablas de los bloques 1 y 4 y del resumen pasan a lista de tarjetas (una fila = una tarjeta) por debajo de 768 px.
- La barra de acción conserva el botón principal y un solo chip «{n} avisos» que abre una hoja inferior con la lista (cada aviso salta a su bloque).
- El estado Resultado: los chips de parámetros en dos columnas; *Editar parámetros* y *Descargar PDF* en fila de ancho completo; el informe se ve a ancho completo y el PDF se descarga igual que en escritorio.

## Accesibilidad

- Cada encabezado de bloque es un `button` con `aria-expanded` y `aria-controls`; los estados llevan icono y texto, no solo color.
- Cada control con `label` asociado; errores bloqueantes con `role="alert"`; avisos y resultado de la consulta con `role="status"`.
- Al saltar a un bloque desde un chip, el foco va al título del bloque; al terminar la consulta, al título del resultado.
- Navegable con teclado: Tab por los campos, Enter y Espacio en los encabezados, Escape cierra los modales.
- Números con separador de miles y la unidad fuera del campo.

# 9 Archivos

| Archivo | Contenido |
|---|---|
| `types.ts` | Tipos de la sección 1.1; campos opcionales en `Section`, `Project` y `MongoProject`; `'AGENT'` en `ProjectView` y en `MongoProject.lastView` |
| `App.tsx` | `'AGENT'` en `VIEW_ORDER`, en la condición de cálculo de `handleNavigate`, en `needsResultsCalc` y en el `switch`; los mapeos y `mergedProject` de 6.1; `handleAgentSave` (6.3) |
| `components/AppHeader.tsx` | Paso 5 en `STEP_DEFINITIONS`, `'AGENT'` en `VIEW_ORDER`, icono nuevo |
| `components/ResultsDisplay.tsx` | Botón *Ir a Recomendaciones* junto a *Exportar* (prop `onOpenAgent`) |
| `components/agent/AgentPanel.tsx` | Contenedor: estados Configurar, Consultando y Resultado; borrador, cola de guardado y salto a bloques |
| `components/agent/AgentHeader.tsx`, `AccordionBlock.tsx`, `ActionBar.tsx` | Cabecera con estado del agente, bloque expandible, barra de acción |

| Archivo | Contenido |
|---|---|
| `components/agent/blocks/ReceivedDataBlock.tsx`, `SectionContextBlock.tsx`, `EquipmentBlock.tsx`, `CostsBlock.tsx`, `BudgetBlock.tsx` | Los cinco bloques de 3.3 a 3.7 |
| `components/agent/result/ResultView.tsx`, `ParamsStrip.tsx`, `QuickAdjust.tsx`, `RunCompare.tsx`, `RunHistory.tsx`, `SummaryTab.tsx` | Estado Resultado (3.9) |
| `components/agent/modals/CopyContextModal.tsx`, `JsonModal.tsx`, `CsvImportModal.tsx`, `ImportResultModal.tsx` | Ventanas compartidas |
| `components/agent/report/AgentReport.tsx` y una pieza por sección (`ReportCover`, `ReportSummary`, `ReportMethod`, `ReportData`, `ReportSection`, `ReportPriority`, `ReportAlerts`, `ReportClosing`, `ReportAnnexes`), `PciGauge.tsx` | Informe en la web (sección 4) |
| `services/agent/agentReportPdf.ts`, `reportText.ts`, `format.ts` | PDF con jsPDF, textos fijos del informe (metodología, bibliografía, firma), formato de números |
| `services/agent/catalog.ts`, `levels.ts` | Catálogo de respaldo (sección 7); `levelForPci`, validación de umbrales, interina por nivel |
| `services/agent/validation.ts` | Estado de cada bloque (ok, aviso, bloquea), mensajes, `summarizeBlock` y efecto de las casillas en los tramos |
| `services/agent/buildAgentInput.ts` | Sección 5 y huella |
| `services/agent/compareRuns.ts` | Comparación entre dos consultas (3.9) |
| `services/agent/csv.ts`, `agentClient.ts`, `defaults.ts` | CSV de costos; cliente con `fetch` (5.1); preferencias del usuario (7.1) |
| `translations/es.ts`, `translations/en.ts` | `stepper.step5` y bloque `agent` completo en ambos idiomas |
| `.env.example`, `README.md` | Documentar `VITE_AGENT_URL` (por defecto `http://localhost:8000`) |
| `server/models/Project.js`, `server/server.js` | Solo en el repositorio de GitHub (6.4) |

## 10 Criterios de aceptación

1. `npx tsc --noEmit` y `npm run build` sin errores nuevos.
2. El menú lateral muestra «5. Recomendaciones» después de «4. Resultados», en español e inglés; el botón *Ir a Recomendaciones* está en Resultados y se deshabilita con explicación si un tramo no tiene PCI.
3. Un proyecto antiguo (sin `agentConfig` ni `context`) abre en Resultados y en Recomendaciones sin errores.
4. La pantalla se ve como las figuras 2 a 9, en tema claro y oscuro, a 1280 px y a 375 px, usando los componentes y tokens de la app.
5. Los bloques son un acordeón con estado (completo, se asumirá, bloquea) y línea de resumen. El contexto vacío da ámbar y no bloquea; umbrales inválidos y ninguna casilla marcada en Equipo bloquean con el motivo en la barra de acción.
6. Los chips de la barra de acción abren y enfocan el bloque que corresponde; los efectos de las casillas (tramos sin opción, interinas) y el número de costos de ejemplo se actualizan al instante.
7. *Copiar este contexto a…* copia todos los campos menos las observaciones a los tramos elegidos.
8. El CSV de costos se importa con vista previa (reconocidas, desconocidas, unidad distinta) y la plantilla descargada trae todos los IDs.
9. El `entrada.json` descargado valida contra `schemas/entrada.schema.json` y el agente lo procesa sin errores, tanto por `POST /api/recommendations` como con `python -m agente todo`.
10. Con el servicio en `localhost:8000`, *Generar recomendaciones* lleva al estado Resultado; con 422 se ven los errores enlazados a su bloque; sin servicio aparecen *Descargar entrada.json* e *Importar resultado…*, que con `analisis.json` + `narrativa.json` da el mismo estado Resultado.
11. El estado Resultado muestra los chips de parámetros, el resumen con las cifras de `analisis`, y la pestaña Informe con todas las secciones de 4.1 con las mismas cifras y textos que `informe-referencia.pdf` para la entrada de ejemplo.
12. **Editar parámetros**, cada chip y el ajuste rápido funcionan: llevan al estado Configurar (o regeneran, en el ajuste rápido) con los valores de la consulta; los campos modificados aparecen «cambiados» con su valor anterior; *Volver a consultar* crea la consulta siguiente y la comparación con la anterior (PCI, financiado, faltante y tramos que cambiaron) es correcta.
13. Con la configuración distinta de la consulta mostrada, aparece la franja «Cambiaste parámetros…»; *Ver resultado v{n}* y el historial permiten volver a una consulta anterior sin llamar al agente, y *Restaurar sus parámetros* recupera su configuración.
14. *Descargar PDF* produce un PDF A4 con las secciones de 4.1, saltos de página por sección y por tramo, y pie con «Página X de Y»; *Imprimir* muestra solo el informe.
15. Tras guardar y recargar la página, el contexto de los tramos, la configuración y el historial siguen ahí, tanto con `localApi` como (si se prueba) con el backend de GitHub con los cambios de 6.4. Con 5 consultas guardadas el proyecto se sigue guardando; si se llena la cuota local, el aviso ofrece borrar la más antigua.
16. Escribir rápido en los campos no pierde ni repite guardados (cola de 6.3).
17. Todo texto visible pasa por `t()` con claves `agent.*` en español e inglés.
18. Sin dependencias nuevas; en el repositorio de GitHub, cambios del backend limitados a 6.4; todo en la rama `feature/recomendaciones`, sin commit, push ni PR hasta que Dk lo confirme, y sin tocar los cambios sin guardar que ya tiene.

## 11 Fuera de alcance

- Reemplazar `agent/` por el paquete del agente y desplegarlo (6.5).
- Importar costos desde Excel; otras monedas; traducir los textos del informe que genera el agente.
- Pavimento rígido (el agente solo cubre flexible).
- Editar las decisiones del agente a mano en la interfaz (cambiar la solución de un tramo): el camino para cambiar el resultado es *Editar parámetros* y volver a consultar.
- Un chat con el agente: no conversa, solo recibe datos y devuelve el resultado.
- Ejecutar obras por etapas para usar el saldo sin asignar (pendiente de decidir en la tesis).

> **AL TERMINAR, INFORMAR**
>
> Archivos creados y modificados, decisiones tomadas donde este documento no alcanzó, resultado de `tsc` y `npm run build`, y lo que no se pudo verificar (por ejemplo, el aspecto visual y el PDF, que revisará Claude en el navegador).